Intelligent Systems with Applications 21 (2024) 200300

Contents lists available at ScienceDirect

Intelligent Systems with Applications

journal homepage: www.journals.elsevier.com/intelligent-systems-with-applications

Learning heuristics for arc routing problems

Muhilan Ramamoorthy *, Violet R. Syrotiuk

School of Computing and Augmented Intelligence, Arizona State University, Tempe, 85281, AZ, USA

ARTICLE INFO

Keywords:
Arc routing problems
AI-driven approaches
Learning heuristics
Machine learning
Deep Q-learning
Combinatorial optimisation

ABSTRACT

Solid waste collection and street sweeping can each be formulated as a *capacitated arc routing problem* (CARP) in which the streets of a city require service by a fleet of vehicles at minimum total cost. Heuristic and meta-heuristic methods have been used to solve CARP because the problem is known to be NP-Hard. In this paper we introduce SPLICE, a novel machine learning framework that learns heuristics to solve routing problems, such as CARP, defined on the edges (rather than vertices) of a non-euclidean graph representation of the problem. SPLICE uses a message passing graph neural network to generate a graph embedding, and deep Q-learning feed-forward layers to learn heuristics to construct a giant tour that results in minimum total cost after applying the splitting procedure, iterating until all tasks are serviced. Compared to the two AI-driven approaches to CARP, SPLICE has a simpler network architecture and does not require a supervised learning method for training. The total costs of the solutions found by SPLICE are from 11-30% lower than those found by a path scanning heuristic and a memetic meta-heuristic algorithm. When trained on small instances, the heuristics learned by SPLICE generalize well, i.e., can be used to solve larger instances without retraining. Due to the *route-first split-second* form of heuristic learned, the SPLICE framework may be applied to solve other variants of routing problems by adapting the splitting procedure.

1. Introduction

Routing problems are among the most widely studied topics in operations research. In arc routing the goal is to visit each *edge* of the graph representing the problem. Applications of arc routing include city services such as street sweeping, snow plowing, mail delivery, solid waste collection, road gritting, and power line inspection, among others (Corberán et al., 2021). Expenditures on such services by public and private entities in the U.S. exceed billions of dollars annually (Corberán & Prins, 2010, Eiselt et al., 1995, Corberán & Laporte, 2015, Dror, 2012), underscoring their importance to the economy.

For city services, the streets of a city can be modelled as a weighted undirected graph $G = (V, E)$ with the streets corresponding to the edges E, and the vertices V to their intersections. In the *capacitated arc routing problem* (CARP) (Golden & Wong, 1981) there is also a *depot* $D \in V$ used to store a fleet of k homogeneous vehicles, each with capacity C. *Tasks* $T \subseteq E$ correspond to the streets requiring service by a vehicle. Each task $t \in T$ has a *demand* $d(t) > 0$; streets not requiring service have zero demand, i.e., for $t \in E \setminus T$, $d(t) = 0$. A *cost* $c(t)$ is associated with a vehicle traversing a street.

The objective of CARP is to find a set of closed routes, one for each vehicle, starting and ending at the depot, of minimum total cost such that:

1. All tasks are serviced.
2. The sum of the demands of tasks on each route does not exceed the vehicle capacity.
3. Every task is in exactly one route.

The *cost of a route* is the cost of traversing the streets serviced in addition to the cost of any *deadheading*, i.e., the cost of traversing any paths that connect streets serviced, as well as paths both to and from the depot. Streets may be traversed by vehicles any number of times when deadheading.

CARP is NP-hard (Golden & Wong, 1981), hence the time to find an optimum solution for most practical instance sizes is intractable. Therefore a number of heuristics and meta-heuristics have been proposed for efficient and close to optimum solutions (Prins, 2013). Meta-heuristics often provide better results than heuristic algorithms, but also require more time to produce them.

Enabled by advances in computer architecture, the performance achieved by *artificial intelligence* (AI) driven approaches to problems

* Corresponding author.
E-mail address: mramamoo@asu.edu (M. Ramamoorthy).

https://doi.org/10.1016/j.iswa.2023.200300
Received 26 July 2023; Received in revised form 20 October 2023; Accepted 9 November 2023
Available online 15 November 2023
2667-3053/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

from computer science (Krizhevsky et al., 2012, He et al., 2016, Ren et al., 2015, He et al., 2017, Redmon et al., 2016) to fields of novel materials and devices (Yi et al., 2023, N. Ma et al., 2023, Sun et al., 2023, P. Ma et al., 2023), has renewed interest in these methods to tackle problems in combinatorial optimisation. Bengio et al. (2020) explore two types of approaches: One is to use *machine learning* (ML) methods to either augment exact methods for solving the problem as a component, or to assist them in their sub-processes, such as predicting the next efficient branch for a branch-and-bound method (Khalil et al., 2016, Balcan et al., 2018). The second is to use ML methods as a stand-alone end-to-end solver to generate a heuristic solution faster, and closer to the optimum, at the time of inference. In this approach, the model learns heuristics by learning the distribution of the parameters of the training instances. It then computes the solution at the time of inference assuming that the new instance is from the same distribution.

Most AI-driven approaches tackle problems represented by a euclidean graph, in which the vertices represent points in the plane, and the edges are assigned lengths equal to the euclidean distance between them. The graph representation of CARP and its variants is *non-euclidean* with all parameters associated only with the *edges*. Hence the existing solution methods cannot be applied directly to arc routing problems. This motivates the need of an AI-driven approach to learn heuristics to solve non-euclidean representations of routing problems.

For this purpose we develop a novel ML framework named SPLICE, a stand-alone end-to-end solver, and apply it to learn heuristics for CARP. SPLICE uses a message passing graph neural network (Gilmer et al., 2017) to generate a graph embedding, and deep Q-learning feed forward layers (Mnih et al., 2015), a form of reinforcement learning, to learn heuristics to construct a giant tour that results in minimum total cost after applying the splitting procedure, iterating until all tasks are serviced. Compared to the two AI-driven approaches to CARP, SPLICE has a simpler network architecture, and it does not require labels or an optimum solution to train the model, speeding both the training and inference phases. SPLICE learns heuristics of the form *route-first split-second* (Lacomme et al., 2004, Ulusoy, 1985). This allows the framework to be applied to variants of the arc and node routing problems by tailoring the splitting procedure to the variant, and is useful for changes to the problem specification commonly encountered by operators of services.

We train SPLICE on structured random CARP instances. We then run the solver, comparing the results of SPLICE with those produced by a path scanning heuristic (Golden et al., 1983), and a memetic meta-heuristic algorithm (Lacomme et al., 2004). SPLICE finds solutions whose total cost range from 11-30% less in comparison. The heuristics learned by SPLICE are also shown to generalize well, taking into account the distribution of the topology and the problem parameters such as cost, demand, and the number of edges.

In summary, the main contributions of the SPLICE framework proposed in this paper include:

- SPLICE learns heuristics to solve *non-euclidean* representations of routing problems using a simple network architecture that does not depend on labels or optimum solutions for training.
- SPLICE produces lower total cost solutions to instances of CARP more efficiently than a heuristic and a meta-heuristic approach.
- By tailoring the splitting procedure of the route-first split second approach for reward estimation, SPLICE can be applied to general routing problems.

The rest of this paper is organized as follows. Work related to CARP is reviewed in §2. The SPLICE framework is introduced in §3, showing how heuristics for CARP are learned. §4 describes how SPLICE is trained, and describes experiments to compare the performance of SPLICE to two other solution approaches. After some discussion of the framework and its limitations in §5, we conclude and propose future work in §6.

2. Related work

2.1. Exact, heuristic, and meta-heuristic approaches

In the *arc routing problem* (ARP), the goal is to visit each edge of the graph representing the problem. The book by Corberán and Laporte (2015) provides a comprehensive discussion of arc routing problems including CARP. The exact methods generally use three approaches to solve ARPs: branch-and-bound based on combinatorial lower bounds, cutting-plane and branch-and-cut methods, and column-generation and branch-and-price methods; see Corberán and Laporte (2015), Belenguer et al. (2013), Muyldermans and Pang (2015).

Golden and Wong, who introduced CARP in 1981 (Golden & Wong, 1981), showed that the problem is NP-Hard. They also proved that finding a 3/2-approximation for CARP is NP-Hard (Golden & Wong, 1981). These facts motivated the use of heuristic and meta-heuristic methods for solving CARP.

Prins (2013) provides a comprehensive review of heuristic and meta-heuristic algorithms proposed for CARP. The classical heuristic algorithms include *Path Scanning* (Golden et al., 1983), among others (Pearn, 1989, Ulusoy, 1985, Wøhlk, 2005), almost all of which are based on heuristics for the *vehicle routing problem* (VRP).

Meta-heuristics are single-solution based such as tabu search (Brandão & Eglese, 2008, Beullens et al., 2003, Usberti et al., 2013, Hertz & Mittaz, 2001), or population-based such as genetic or evolutionary algorithms. The memetic algorithm (Lacomme et al., 2004) is a population-based meta-heuristic using evolutionary based methods; it is a hybrid meta-heuristic using local search in the place of mutation. Recently, swarm-based meta-heuristics (Marinaki et al., 2023, Hamza et al., 2023), a kind of population-based meta-heuristic, have also been developed for node routing problems.

2.2. AI-driven approaches

As early as 1985, Hopfield networks (Hopfield & Tank, 1985), elastic nets (Durbin & Willshaw, 1987), and self organizing maps (Fort, 1988), were applied to problems in combinatorial optimisation; see Smith (1999) for an overview. At the time, the solutions produced were not close enough to the optimum nor practical enough to be applied to real applications.

More powerful computers renewed the interest in exploring AI-driven approaches for such problems (Bengio et al., 2020). By the mid 2010s, recurrent neural network (RNN) based sequence-to-sequence networks (Sutskever et al., 2014), and the attention mechanism (Bahdanau et al., 2015), were successful in machine translation and other applications. Vinyals et al. (2015) used a variation of long short-term memory (LSTM) sequence-to-sequence networks, called pointer networks, applying their approach to the two-dimensional euclidean travelling salesman problem (2D-TSP), convex hull, and Delaunay triangulation. The model generalized well for problems up to 40 nodes but not beyond.

Pointer networks have become the basis of much follow-on work. Bello et al. (2016) applied pointer networks but used the actor-critic policy gradient method (Williams, 1992) to train the network. With this method, closer to optimum results for up to 100 nodes for 2D-TSP were obtained. The model was also used to solve the knapsack problem obtaining optimal results for up to 200 nodes.

Nazari et al. (2018) extended pointer networks to VRP using an embedding layer instead of an RNN but the same LSTM based network with the attention mechanism. They added the asynchronous advantage actor-critic (A3C) method (Mnih et al., 2016) for training and were able to achieve a smaller optimality gap.

Based on their previous work `structure2vec` (Dai et al., 2016), Khalil et al. (2017) used a graph embedding neural network for the problems of maximum vertex cover, maximum cut, and 2D-TSP. They

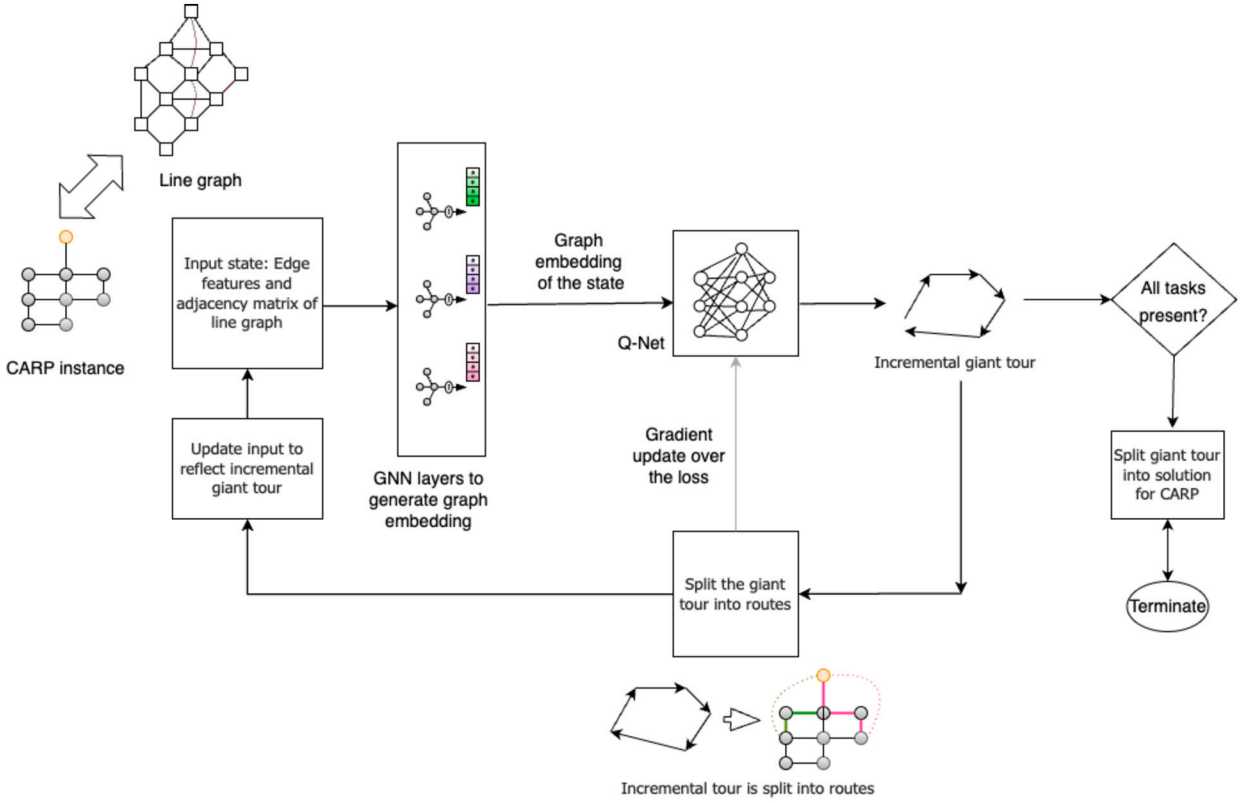


Fig. 1. Block diagram of the SPLICE framework to learn heuristics for solving arc routing problems. It incrementally constructs a solution to an instance represented by a graph $G = (V, E)$ drawn from D, with tasks $T \subseteq E$. It uses a message passing graph neural network to generate a graph embedding, and deep Q-learning feed forward layers to learn heuristics to split an incremental giant tour into routes for vehicles. It iterates until all tasks are serviced, i.e., the giant tour includes all tasks. See §3 for a description of each block.

used Q-learning (Mnih et al., 2015) for training because it is more sample efficient than the policy gradient methods (Williams, 1992). The data sets included up to 300 nodes for training and, on testing, achieved a cost within 0.07% of the optimum.

To further reduce the optimality gap, Kool et al. (2019) and Deudon et al. (2018) used multi-head attention networks (Vaswani et al., 2017), while Joshi et al. (2019) used a spectral-based graph convolution network (GCN) (Bresson & Laurent, 2017). Improvement heuristics and local search methods have also been explored (Chen & Tian, 2019, Barrett et al., 2020, de O. Costa et al., 2020, Wu et al., 2019, Gao et al., 2020) for this purpose.

More recently, approaches to handle the complexity of the combinatorial space include exploiting symmetries by using parallel multiple rollouts learning heuristics (Kwon et al., 2020), formulating the problem as a value function and using a policy iteration algorithm for solving it (Delarue et al., 2020), and using the policy derived from a neural network (Kool et al., 2022).

Yaddaden et al. (2022) used the route-first split-second procedure in their method for solving capacitated VRP with a policy gradient method to learn heuristics. Their work is a concurrent with but independent of our work (Ramamoorthy, 2022).

2.2.1. *Approaches for non-Euclidean graphs*

For node routing problems there are well known methods to learn the graph representation. These include random walks (Lovász, 1993), and node embedding methods such as `node2vec` (Grover & Leskovec, 2016). As arc routing problems are instead defined on the edges of non-euclidean graphs, methods are required to transform the input to encode edge features to learn the graph representation, and to learn heuristics.

To the best of our knowledge, there are two studies related to learning heuristics for CARP. Li and Li (2019) consider CARP as a set-to-sequence problem and use a policy gradient to learn the heuristics. In their network architecture, they first use a graph convolution network (Defferrard et al., 2016) to generate embeddings of the edge set. Then, two pointer networks are used: One outputs a sequence from the edge set embeddings, and the second determines the direction of each task in the generated sequence.

Hong and Liu (2022) instead consider CARP as a sequence-to-sequence problem and use only a single `seq-2-seq` based pointer network. Their method has three stages in the training phase. First, the input sequence is constructed by sorting the edges based on the distances between the edges. In the second stage a random-walk based method, `node2vec` (Grover & Leskovec, 2016), is used to generate the embedding for the edges in the input sequence. A pointer network in the final stage generates the sequence that corresponds to the solution. A supervised learning method is used to train the model.

We next describe our SPLICE framework for solving CARP. It has a simpler network architecture compared to that used in Li and Li (2019), and does not need a supervised learning method for training as used in Hong and Liu (2022), speeding both the training and inference phases of SPLICE.

3. The SPLICE framework for learning heuristics

SPLICE is an AI-driven framework that learns heuristics for solving instances of CARP represented by graphs drawn from a distribution D. Fig. 1 shows a block diagram of the framework. SPLICE is applied to training instances to learn a model that can then be applied to yield solutions for test instances. Training and test instances are represented by a graph $G = (V, E)$ drawn from D, with tasks $T \subseteq E$.

At a high level, in each step $1 \leq i \leq |T|$ of the loop, a message passing graph neural network (GNN) is used to learn the graph embedding, followed by additional feed forward layers to learn the functional

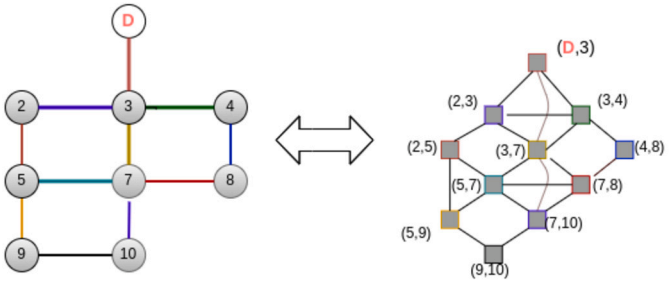


Fig. 2. A graph G representing an instance of CARP (left), and its line graph transformation $G_{\mathcal{L}}$ (right). D is the depot. Each edge in G corresponds to a vertex $G_{\mathcal{L}}$. For example, the edges $(D,3)$ and $(3,4)$ in G correspond to the vertices $(D,3)$ and $(3,4)$ in $G_{\mathcal{L}}$. If (u,v) and (v,w) are two edges in G that share a common end-point v, then there is an undirected edge from vertex (u,v) to vertex (v,w) in $G_{\mathcal{L}}$. Continuing the example, because the edges $(D,3)$ and $(3,4)$ in G are both incident on vertex 3, there is an edge between vertices $(D,3)$ and $(3,4)$ in $G_{\mathcal{L}}$.

approximation of a solution through deep Q-learning. The adjacency matrix of the line graph $G_{\mathcal{L}}$, a transformation of G, together with the edge features form the input state for the GNN. The output of the GNN in step i is a graph embedding. The action corresponding to the maximum quality function value (or, Q-value) corresponds to the selection of a task, which is appended to the incremental giant tour.

An incremental giant tour at step i is an ordered sequence of i tasks that does not consider the capacity limit of the vehicles. Only when an incremental tour is split into routes for vehicles does it take into account their capacity C and other constraints of CARP. The meta-algorithm learns route-first split-second heuristics through deep Q-learning. It estimates a reward for the incremental solution and updates the gradient over the loss. The state of the input is then updated to reflect the results of step i for use in the next step.

The loop iterates $|T|$ times, after which all tasks are included in the giant tour. When the giant tour is split into routes and spliced together, a solution for the CARP instance represented by G is found. This process motivates the name of our framework.

We now describe each block in the SPLICE framework in more detail.

3.1. Input feature and state representation

In CARP the problem instance is represented by a weighted undirected graph $G=(V,E)$, together with the set of tasks $T \subseteq E$. Each task $t \in T$ has a traversal *cost* $c(t)$ and a *demand* $d(t)$. Because CARP is defined on the edges of G and not its vertices, node embedding methods cannot be applied directly. Therefore G is transformed into a line graph to make it more amenable to generate a graph embedding by a GNN.

The *line graph* $G_{\mathcal{L}}=(V_{\mathcal{L}},E_{\mathcal{L}})$ transformation of $G=(V,E)$ has vertices that correspond to the edge set E of G, i.e., $V_{\mathcal{L}}=E$. If (u,v) and (v,w) are two edges in E that share a common end-point v, then there is an undirected edge from vertex $(u,v) \in V_{\mathcal{L}}$ to vertex $(v,w) \in V_{\mathcal{L}}$ in the edge set $E_{\mathcal{L}}$ (Gross et al., 2018, p. 20). Fig. 2 shows an example of a graph representation of a CARP instance and its corresponding line graph.

While the line graph captures the edges E of the CARP instance and their connectivity, it does not capture the other parameters of the problem. To complete the representation of the problem, for each edge $e=(u,v) \in E$, a 7-tuple feature vector $\langle u,v,c(e),d(e),present,first,last\rangle$ is introduced. The vector includes the endpoints of the edge, u and v, the cost $c(e)$ associated with traversing edge e, the demand $d(e)$ of edge e (recall that if $e \in E \setminus T$ then $d(t)=0$), and three indicator variables. The value of the indicator variables depends on the incremental giant tour in iteration i. *present* is true if $e \in T$ and e is in the incremental giant tour, and false otherwise. Similarly *first* and *last* are true if $e \in T$ and e is the first, respectively last, edge in the incremental giant tour. The indicator variables of all edges are initialized to false. As tasks are added incrementally to the giant tour, the tuples are updated to reflect the change in state.

3.2. Message passing GNN for learning the graph embedding

In order for subsequent layers to learn the meta-algorithm heuristic function (Xu et al., 2019), the input state must be transformed into a suitable graph embedding representation. *Graph neural networks* (GNNs) are effective to learn and represent non-euclidean graphs. While many different methods are known, in SPLICE we use a message passing GNN (Gilmer et al., 2017) for this purpose.

A message passing GNN learns an embedding by iteratively aggregating the information of the neighbours of a vertex and then using a non-linear transformation of the aggregated values with itself. In each step i of SPLICE, there are k layers of message passing in the GNN. In general, the embedding $h_u^{i,k}$ of vertex u in step i after k layers of message passing is given by Hamilton (2020):

$$h_u^{i,j}=\text{Update}^{j-1}\left(h_u^{i,j-1},\text{Aggregate}^{j-1}_{v\in N(u)}(h_v^{i,j-1})\right), \tag{1}$$

for $j=1,\ldots,k$, and where $N(u)$ are the neighbours of u. The Update$(\cdot)$ and Aggregate$(\cdot)$ functions are any differentiable functions that can be implemented in a neural network. Usually a non-linear function is used for the Update$(\cdot)$ function. The aggregation function should be permutation invariant to remove dependence on the input order. Each layer of the GNN extends the learning of neighbours through aggregation.

We now tailor Equation (1) in the context of CARP in the SPLICE framework. The vertices of $G_{\mathcal{L}}$ correspond to edges of the CARP instance, and $h_u^{i,k}$ represents the input features for the edge u at step i after k iterations of GNN message passing. Here, $h_u^{0,0}$ corresponds to the 7-tuple feature vector initialized for each edge. The adjacency matrix of the line graph $G_{\mathcal{L}}$ is used to determine the neighbourhood $N(u)$ of u, for aggregation of messages in the GNN.

Initially the giant tour $\mathcal{G}_0=\langle\rangle$ is empty, and $\mathcal{T}_0=T$, the set of all tasks in the CARP instance. The neighbours $N(u)$ of a task u are the tasks v reachable from u, i.e., v such that $(u,v) \in V_{\mathcal{L}}$. The *rectified linear unit* (ReLU) function (Nair & Hinton, 2010) is used as the Update$(\cdot)$ function and Aggregate$(\cdot)$ is a summation of the embeddings. We do not normalize after aggregation to avoid any loss of information. Equation (1) in the context of CARP in SPLICE to update the graph embedding for a task u in the ith step is rewritten as:

$$h_u^{i,j}=ReLU\left(\theta_1 h_u^{i,j-1}+\theta_2\sum_{v\in N(u)}h_u^{i,j-1}\right), \tag{2}$$

for $j=1,\ldots,k$, where θ_1 and θ_2 are the model parameters of the embedding that are learned. The number k of iterations used by the GNN to compute the embedding is bounded by the half the graph diameter because after this number of iterations, all vertices of the graph are encompassed; more iterations lead to over-smoothing (Hamilton, 2020).

The subsequent layers of SPLICE use functional approximation to estimate the evaluation function $Q(\mathcal{G}_{i-1},u,\Theta)$ to measure the quality of vertex u with respect to the incremental giant tour $\mathcal{G}_{i-1}$. These are the action-value pairs for the current input state:

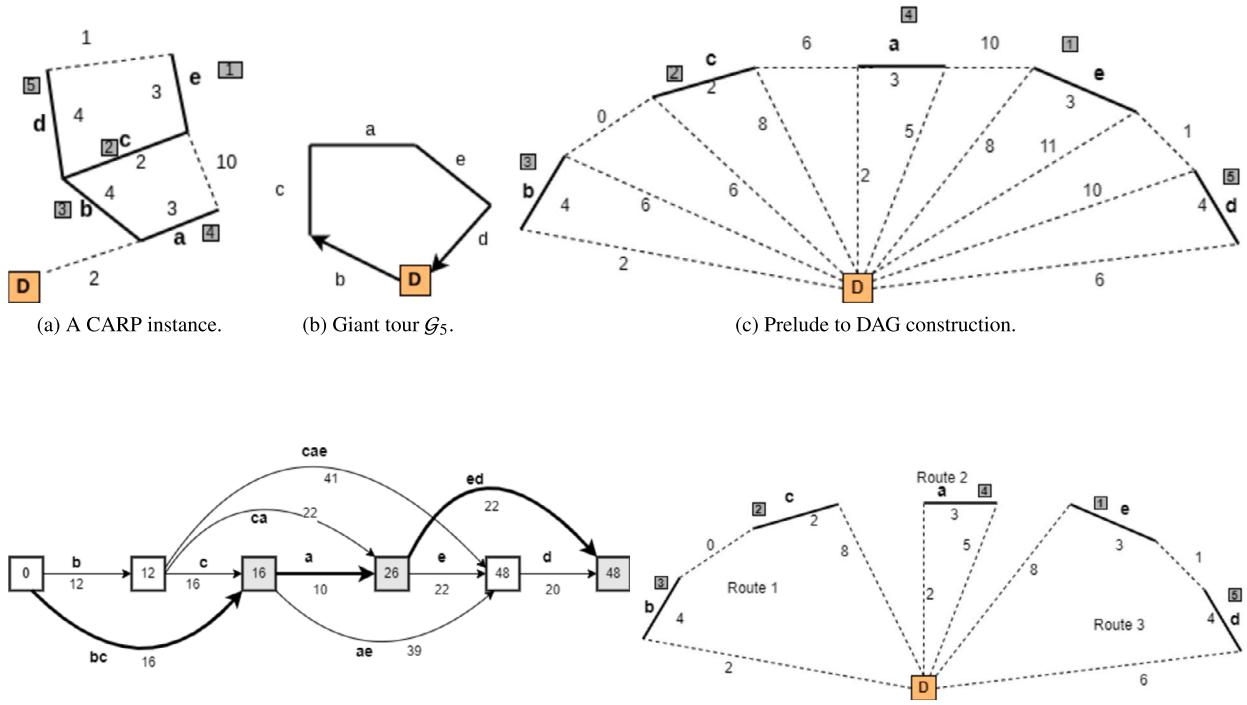
$$Q(\mathcal{G}_{i-1},u,\Theta)=ReLU(\theta_3\, h_u^{i,k}), \tag{3}$$

where $\Theta=\langle\theta_1,\theta_2,\theta_3\rangle$ is the vector of model parameters.

An ϵ-greedy strategy (Sutton & Barto, 2018) is used to aid exploration during the training phase. It selects the action with the maximum embedding value with probability $1-\epsilon$. This corresponds to the selection of a task u of highest quality Q value:

$$u=\arg\max_{v\in\mathcal{T}_{i-1}} Q(\mathcal{G}_{i-1},v,\Theta) \tag{4}$$

from all remaining tasks to be completed, $\mathcal{T}_{i-1}$. With probability ϵ, the strategy selects an action (task) chosen uniformly at random from $\mathcal{T}_{i-1}$.



(a) A CARP instance. (b) Giant tour $\mathcal{G}_5$. (c) Prelude to DAG construction.

(d) Auxiliary DAG taking into account capacity, $C = 7$. (e) Routes resulting from splitting the giant tour.

Fig. 3. An example of the splitting procedure for a CARP instance based on the route-first split-second heuristic: (a) A sample CARP instance with five tasks $T = \{a, b, c, d, e\}$ with the demand for each indicated inside a square. Each edge also has a traversal cost. (b) A giant tour $\mathcal{G}_5 = \langle b, c, a, e, d \rangle$ containing all tasks. (c) For each task in the giant tour, the cost of the shortest path from the depot to the task returning back to the depot including dead-heading is indicated. In addition to the cost of the shortest path between tasks via dead-heading is indicated. (d) The auxiliary DAG taking into account capacity, $C = 7$, of the vehicles. (e) The routes resulting from splitting the giant tour. See §3.3.1 for the details on figures (d) and (e).

The vertex u selected is appended to the incremental giant tour, i.e., $\mathcal{G}_i = \langle \mathcal{G}_{i-1}, u \rangle$, an ordered list of tasks. In addition, u is removed from the set of tasks that remain to be serviced, i.e., $\mathcal{T}_i = \mathcal{T}_{i-1} \setminus u$.

3.3. Deep Q-learning for learning the heuristics

For each task u, the model parameters $\Theta = \langle \theta_1, \theta_2, \theta_3 \rangle$ of the quality evaluation function $Q(\mathcal{G}_i, u, \Theta)$ are learned using deep Q-learning (Mnih et al., 2013), a form of reinforcement learning. We chose to use Q-learning (Watkins, 1989) because it is sample efficient and is more stable than other methods (Sutton & Barto, 2018). SPLICE combines n-step fitted Q-iteration (Riedmiller, 2005) with deep Q-learning (Mnih et al., 2015) to learn the parameters.

The target value y is the sum of the reward and the maximum action-value of the next state scaled by discount factor γ (Sutton & Barto, 2018), in step i:

$$y = \text{reward}(\mathcal{G}_i) + \gamma max_{u' \in \mathcal{T}_i} Q(\mathcal{G}_i, u', \Theta), \tag{5}$$

where the reward function is defined in Equation (7) and described in §3.3.1.

The mean squared loss is defined as the square of the difference between the target value y and the current state value given by:

$$loss = (y - Q(\mathcal{G}_i, u, \Theta))^2. \tag{6}$$

Fitted Q-iteration uses *experience replay*, performing batch updates from a replay memory at the gradient steps rather than after each sample; a sample consists of the tuple with the current state, the action taken in the current state, the reward received for that action, the resulting state, and the action corresponding to highest Q value (which may be different due to the ϵ-greedy strategy). This helps to mitigate the correlation issues faced by performing updates of loss from sequential samples (Mnih et al., 2015).

In step i, the giant tour $\mathcal{G}_i$ is a sequence of i tasks. That is, we have generated a tour ("route-first") and now we must split the tour into a set of closed routes ("split-second") to solve CARP for $\mathcal{T}_i$. Then a reward is estimated for this solution, and the loss is propagated; these are discussed next.

3.3.1. Splitting a giant tour into routes and reward estimation

A giant tour must be split into individual routes respecting the capacity limit C of the vehicles and other CARP constraints. Methods for splitting a giant tour into routes are well studied in arc routing (Ulusoy, 1985, Vidal, 2016, Prins et al., 2014, 2009).

In SPLICE, the splitting procedure is based on the route-first split-second heuristic proposed by Ulusoy (1985). First, an auxiliary directed acyclic graph (DAG), is constructed based on the giant tour $\mathcal{G}_i$. Each arc in the DAG represents a route traversing the same sequence of tasks in the giant tour subject to the vehicle capacity limit, with its cost representing the distance starting from and ending at the depot. The shortest path from the source to the sink in the DAG gives the routing solutions.

Fig. 3a illustrates the splitting procedure for a sample CARP instance with five tasks $T = \{a, b, c, d, e\}$ with the demand for each indicated inside a square; recall non-required edges have zero demand. Each edge also has an associated traversal cost. Fig. 3b shows a giant tour $\mathcal{G}_5 = \langle b, c, a, e, d \rangle$ containing all tasks. Fig. 3c shows, for each task in the giant tour $\mathcal{G}_5$, the cost of the shortest path from the depot to the task returning back to the depot including dead-heading, in addition to the cost of the shortest path between tasks via dead-heading, if required. In this example, task c is adjacent to task b in $\mathcal{G}_5$ hence no dead-heading is required to service task c after servicing task b.

From this graph, the auxiliary DAG is constructed. While $\mathcal{G}_i$ has i vertices, the DAG has one additional vertex for the depot. The vertices in the DAG correspond to the cost of servicing each subsequence t_k, $k \leq j, j = 1, \ldots, i$ of tasks in $\mathcal{G}_i = \langle t_1, t_2, \ldots, t_i \rangle$, while the arcs are labelled by the feasible subsequences of tasks. Feasibility is dictated by sequence

order in the giant tour and capacity constraints of the vehicle. All routes in CARP start and end at the depot, hence the cost of the first vertex is zero.

Suppose that the vehicle capacity $C = 7$. Because the first task in $\mathcal{G}_5$ is b, all arcs out of the depot must start by servicing task b. As Fig. 3c shows, servicing task only b has a cost of 12, hence the auxiliary graph has a vertex with cost 12 for servicing task b. However the demand of task b is only 3, i.e., $d(b) = 3$, leaving a capacity $C = 7 - 3 = 4$ in the vehicle. This provides the possibility to service another task in the route. Only task c is feasible. The route bc shown as arc bc with cost 16 in the DAG. It is obtained from Fig. 3c by including the cost of tasks b and c, the cost of the deadheading between them, and cost of dead-heading from the depot to b, and from c to the depot: $4 + 2 + 0 + 2 + 8 = 16$. The route bc cannot be extended because the remaining capacity of the vehicle is now $C = 4 - 2 = 2$ and the demand 4 of the next task in the sequence, a, exceeds the vehicle's remaining capacity.

Each vertex in the auxiliary graph is extended in all feasible ways subject to task order and capacity constraints. The last vertex in the DAG is then the total cost of servicing all tasks in $\mathcal{G}_i$. After the construction of the DAG is complete, the shortest path from the depot to the last vertex is the minimum cost of servicing all tasks.

Returning to the example, Fig. 3d shows a shortest path with three arcs in bold with a total cost of 48 for $\mathcal{G}_5$. Each arc corresponds to a closed route that includes the tasks serviced by a vehicle following that route. In this example, there are three routes from the depot with one servicing task b followed by c, a second servicing only task a, and a third servicing task e followed by d. These tasks are spliced together to give a solution for $\mathcal{G}_5$.

After splitting and computing the total cost for $\mathcal{G}_i$ at step i, we compute the reward for the current action of appending task u to $\mathcal{G}_i$ as:

$$reward(\mathcal{G}_i) = -\left(totalcost(\mathcal{G}_i) - totalcost(\mathcal{G}_{i-1})\right), \tag{7}$$

where the $totalcost(\mathcal{G}_0) = 0$.

The input features of all edges are then updated for the next step of the framework.

The run-time and space complexity of the splitting procedure is $O(|T|^2)$, where $|T|$ is the number of tasks. The splitting procedure described can be performed without constructing the DAG explicitly (Lacomme et al., 2004); this reduces the space complexity to $O(|T|)$.

A giant tour of minimum cost that is split into a routing solution does not necessarily result in a solution of minimum total cost. This is one way in which SPLICE differs from the learning methods for solving the TSP. SPLICE does not learn to construct the tour of minimum cost. Instead, it learns a tour that yields minimum total cost upon optimal splitting.

3.4. The SPLICE algorithm

The SPLICE framework is now summarized using pseudocode in Algorithm 1. In line 3, the algorithm iterates for N episodes to train the network. Each episode trains using a single graph.

In line 4, a graph representation G of a CARP instance is selected at random from a training data set with distribution $\mathcal{D}$. G is transformed into a line graph $G_{\mathcal{L}}$, and the input features for each edge are collected.

The outer loop of the algorithm in line 9, which corresponds to the outer loop in the block diagram in Fig. 1, iterates for $|T|$ steps. In each step, a message passing GNN generates a graph embedding (line 10). The embedding is used as input to the Q-Net, the block representing layers between the GNN layers and the output in Fig. 1. Using the ϵ-greedy strategy, a task u is selected (line 12) and added into the giant tour. Then after splitting the giant tour, a reward is computed for the solution (line 18) and stored in the experience replay memory (line 19). In fitted Q-iteration, loss is not performed on sequential samples but instead performed on batches at a gradient step to avoid correlation (line 22). The state is then updated (line 23) for the next step of the outer loop.

Algorithm 1: The SPLICE framework.

Input : CARP instance $G = (V, E)$, $T \subseteq E$, and $c(e)$ and $d(e)$ for each $e \in E$
Output: SPLICE model, Θ

1 Initialize experience replay memory M capacity
2 Set hyper-parameters as given in Table 1
3 **for** $episode \leftarrow 1$ **to** N **do**
  /* Select a CARP instance */
4 Select a graph G at random for training from distribution $\mathcal{D}$
5 Transform G into line graph $G_{\mathcal{L}}$
6 Initialize input state: Adjacency matrix of $G_{\mathcal{L}}$ and feature vector for each edge
7 Initialize incomplete tasks $\mathcal{T}_0 \leftarrow T$
8 Initialize giant tour $\mathcal{G}_0 \leftarrow \langle\rangle$
  /* Use $\mathcal{N}$-step Q-learning to learn the parameters */
9 **for** $i \leftarrow 1$ **to** $|T|$ **do**
10 Use message passing to learn the graph embedding
  /* Use ϵ-greedy strategy to select next task, u */
11 Choose probability p uniformly at random
12 **if** $p < \epsilon$ **then**
13 $u \leftarrow$ choose task uniformly at random from $\mathcal{T}_{i-1}$
14 **else**
15 $u \leftarrow \arg\max_{v \in \mathcal{T}_{i-1}} Q(\mathcal{G}_{i-1}, v, \Theta)$
  /* Append u to incremental giant tour; update task set */
16 $\mathcal{G}_i \leftarrow \langle \mathcal{G}_{i-1}, u \rangle$
17 $\mathcal{T}_i \leftarrow \mathcal{T}_{i-1} \setminus u$
18 Split $\mathcal{G}_i$ in routes; compute reward($\mathcal{G}_i$)
19 Add sample into replay memory M
  /* Use fitted Q-iteration with experience replay */
20 **if** $i \bmod \mathcal{N} == 0$ **then**
21 Obtain a random sample batch from replay memory, $B \overset{iid}{\sim} M$
22 Perform gradient update of Θ over loss (Equation (6)) for the batch B
  /* Update state */
23 update feature vectors for each edge
24 return Θ

After training, the algorithm returns the model Θ which can then be used to apply the heuristics learned to obtain a solution to a new CARP instance from distribution $\mathcal{D}$ in a similar manner.

4. Experimentation and results

4.1. Training instances

Arc routing problems are often defined on a city's road network. Commonly these are grid-shaped, hence the graph representation of the road network has a grid-like topology. We create a distribution $\mathcal{D}$ of grid-like topologies, deleting at most k edges from a grid at random, to create variations in topology and in vertex degree. These were then augmented with costs and demands to generate a CARP instance.

To generate a training instance, first a graph $G = (V, E)$ in the form of an $m \times n$ grid of vertices with $2mn - m - n$ edges is generated; vertex 1 is designated as the depot. At most k edges are deleted at random. As edges are deleted, if a vertex becomes isolated it is deleted from the instance; if the depot becomes isolated the instance is abandoned and generation is restarted. Then, costs and demands are assigned to the edges from a distribution according to the specific arc routing problem. For CARP, traversal costs are assigned to all edges, with non-zero demand assigned to all tasks in $T \subseteq E$ and zero demand assigned to edges in $E \setminus T$. The *Capacitated Chinese Postman Problem* (CCPP) is a variant of CARP in which $T = E$. The vehicle capacity C is the same across instances.

For training SPLICE, each instance for CARP and CCPP starts as a 5×4 grid. Then, $k = 2$ edges are deleted from the grid at random yielding 29 edges. The cost of each edge is varied uniformly at random from

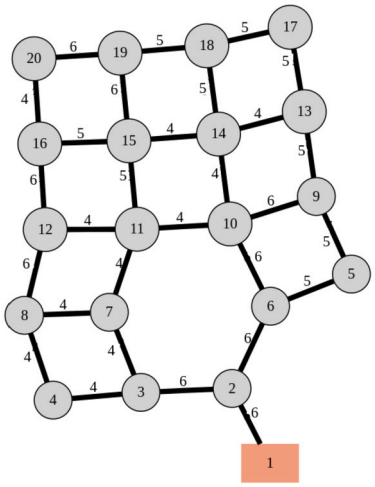


Fig. 4. An example 5×4 training instance generated for CARP with $k = 2$ edges deleted at random. This yields a graph with 29 edges. The edge costs are selected uniformly from $[4, 6]$; the demands are not indicated. Vertex 1 is the depot.

the range $[4, 6]$; the cost is not varied much to ensure that the topology of the graph remains planar. The set of tasks for the instance is initialized to the edge set, i.e., $T = E$. The demand of each edge is selected uniformly at random from $[0, 5]$ for a CARP instance. If an edge e has demand zero, $d(e) = 0$, then $T = T \setminus e$. Fig. 4 shows one of the instances generated for CARP by this method. For a CCPP instance, the demand of each edge is selected uniformly at random from $[1, 6]$; thus $T = E$ as required.

For each of CARP and CCPP, a total of 20,000 distinct instances were generated. These were divided into training, validation, and testing data sets with a ratio of 80:10:10, respectively, i.e., 16,000:2,000:2,000.

4.2. Hyper-parameter tuning and selection

The hyper-parameter settings used in many general reinforcement learning methods for control tasks fit our experiments without the need for extended tuning. The metrics of loss and the running average of the route cost was monitored. From observations of the stability of these metrics, 10,000 training episodes were found to suffice. An embedding dimension of five was found to be adequate for models of CARP instances with both 15 and 29 edges with a 7-tuple feature vector. Extending the embedding dimension beyond five does not appear to provide any advantage. The replay buffer capacity plays a role in how long the transitions are retained and used for learning (Zhang & Sutton, 2017, Fedus et al., 2020). We found a buffer capacity of 4,000 fit our architecture, input problem size, and the number of episodes. The learning rates, and the learning decay rate for the Adam optimizer (Kingma & Ba, 2015), were set to the standard values and did not require any further tuning. The hyper-parameter settings used in SPLICE for training, validation, and testing are summarized in Table 1.

The Pytorch (Paszke et al., 2019) framework was used for creating the neural network layers in the `python` programming language. The `Networkx` (Hagberg et al., 2008) package was used for generating instances, and for line graph transformations.

4.3. Experimental set-up: training

We performed two experiments to evaluate the inference capability of SPLICE. The first was to solve instances of CCPP, a variant of CARP in which all the edges are required, i.e., $T = E$. The second experiment was to solve instances of CARP. For each experiment the model was trained on 10,000 of the 16,000 training instances generated for the problem variant; see §4.1. In both these experiments the vehicle capacity was set to $C = 16$. The test instances were selected at random from the 2,000 of 20,000 data sets reserved for testing for each problem variant.

Table 1
Hyper-parameter settings used in SPLICE.

Hyper-parameter	Value
Number of episodes	10,000
Embedding dimension	5
Discount factor, γ	0.9
Number of layers	5
Learning rate, α	0.005
Learning decay rate, η	0.99998
ϵ in ϵ-greedy strategy	0.1
ϵ decay rate	0.0006
n-step reward update, $\mathcal{N}$	2
Replay buffer memory capacity	4,000
Batch size	16

The purpose of the third experiment was to evaluate the generalisation capability of the model. In this experiment we trained the SPLICE model on smaller CARP instances and evaluated how it performed on larger instances. The smaller training instances were generated in the same manner described in §4.1 except that the starting grid was 4×3; $k = 2$ edges were removed, leaving a graph with 15 edges. Similarly, 20,000 distinct instances were generated for generalisation with the same ratio of 80% of them used for training. The instances used in testing were drawn from the 2,000 reserved for testing from the CARP data set on 29 edges. For both the 15 and 29 edge instances, the vehicle capacity was set to $C = 8$.

For all the three experiments the average loss decreased continuously during training and reached the lowest value at around episode 9,500 and subsequently remained flat.

We compare the results of SPLICE on test instances with those produced by the *path scanning* (PS) heuristic (Golden et al., 1983), and the *memetic algorithm meta-heuristic* (MA) (Lacomme et al., 2004). The hyper-parameters of MA are set to 300 generations, a population size of 100, and the probability of cross-over and mutation to be certainty.

4.4. Results and analysis

Inference was performed on the test instances without further learning or updating of the gradients.

4.4.1. Results of the CCPP experiment

Ten instances were selected uniformly at random from the 2,000 test instances in the CCPP data set. For each instance, Table 2 gives the total cost of the solutions achieved by SPLICE (S) and path scanning (PS), the percentage decrease in total cost of the SPLICE solution compared to the path scanning solution $(1 - \frac{S}{PS}) \times 100\%$, the total cost of the solution achieved by the memetic algorithm (MA), and the percentage decrease in total cost of the SPLICE solution compared to the memetic algorithm solution $(1 - \frac{S}{MA}) \times 100\%$. Also given is the average decrease in total cost over the ten instances for the heuristic and the memetic algorithm. The total cost for these ten instances is also plotted in Fig. 5.

As Table 2 and Fig. 5 show, the total costs of the solutions achieved by SPLICE were lower than the solutions achieved by path scanning and the memetic algorithm for all ten CCPP instances. In fact, the SPLICE solutions are 29.03% lower on average than the path scanning solutions, and 23.43% lower than the memetic algorithm solutions. The memetic algorithm outperforms path scanning on all these instances; this is expected as meta-heuristics generally outperform heuristics (Prins, 2013). This also explains why SPLICE outperforms path scanning by almost 6% more than the memetic algorithm.

Table 2
The total cost for ten instances of CCPP selected at random from 2,000 test instances for SPLICE (S), path scanning (PS), and the memetic algorithm (MA). In addition, the percentage decrease in total cost of the SPLICE solution compared to that of path scanning (%↓ $\frac{S}{PS}$), and compared to that of the memetic algorithm (%↓ $\frac{S}{MA}$).

	Total Cost				
Instance	S	PS	%↓ $\frac{S}{PS}$	MA	%↓ $\frac{S}{MA}$
29E_19160	228	304	25.00	296	22.97
29E_12689	231	329	29.79	305	24.26
29E_8479	234	333	29.73	312	25.00
29E_17805	238	326	26.99	308	22.73
29E_12410	228	366	37.70	330	30.91
29E_5463	241	365	33.97	316	23.73
29E_4942	249	387	35.66	343	27.41
29E_9657	224	270	17.04	254	11.81
29E_12271	239	341	29.91	329	27.36
29E_11712	249	330	24.55	304	18.09
		Avg:	29.03	*Avg:*	23.43

Table 3
The total cost for ten instances of CARP selected at random from 2,000 test instances for SPLICE (S), path scanning (PS), and the memetic algorithm (MA). In addition, the percentage decrease in total cost of the SPLICE solution compared to that of path scanning (%↓ $\frac{S}{PS}$), and compared to that of the memetic algorithm (%↓ $\frac{S}{MA}$).

	Total Cost				
Instance	S	PS	%↓ $\frac{S}{PS}$	MA	%↓ $\frac{S}{MA}$
29E_2185	219	233	6.01	216	-1.39
29E_5093	229	282	18.79	264	13.26
29E_155	238	287	17.07	282	15.60
29E_9689	234	287	18.47	285	17.89
29E_7372	228	279	18.28	255	10.59
29E_15979	235	266	11.65	253	7.11
29E_14782	226	288	21.53	276	18.12
29E_11832	228	313	27.16	285	20.00
29E_776	217	232	6.47	208	-4.33
29E_4620	211	236	10.59	236	10.59
		Avg:	15.60	*Avg:*	10.75

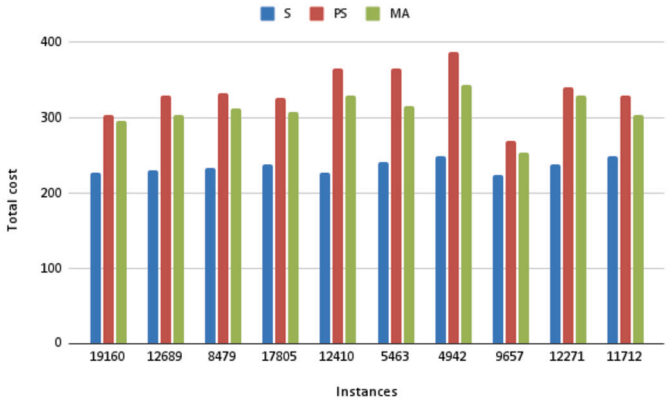


Fig. 5. Total cost of the solutions for the ten instances of CCPP from Table 2 with 29 edges and $C = 16$ for SPLICE (S), the path scanning heuristic (PS), and the memetic algorithm meta-heuristic (MA).

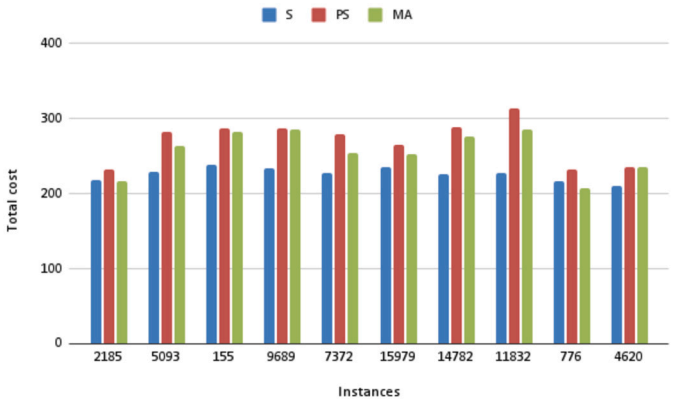


Fig. 6. Total cost of the solutions for the ten instances of CARP from Table 3 with 29 edges and $C = 16$ for SPLICE (S), the path scanning heuristic (PS), and the memetic algorithm meta-heuristic (MA).

4.4.2. Results of the CARP experiment

Ten instances were selected uniformly at random from the 2,000 test instances in the CARP data set. For each of these instances, Table 3 gives the total cost of the solutions achieved by SPLICE (S) and path scanning (PS), the percentage decrease in total cost of the SPLICE solution compared to the path scanning solution $(1 - \frac{S}{PS}) \times 100\%$, the total cost of the solution achieved by the memetic algorithm (MA), and the percentage decrease in total cost of the SPLICE solution compared to the memetic algorithm solution $(1 - \frac{S}{MA}) \times 100\%$. Also given is the average decrease in total cost over the ten instances for the heuristic and the memetic algorithm. The total cost for these ten instances is also plotted in Fig. 6.

As Table 3 and Fig. 6 show, for CARP, the solutions achieved by SPLICE have lower total cost than those found by path scanning for all ten instances. On average, SPLICE outperforms path scanning by 15.60%. The memetic algorithm, however, achieved better solutions than SPLICE for two of the ten instances, by a small margin (1.39% and 4.33%). But on average, SPLICE also outperforms the memetic algorithm by almost 11%. Again, because the memetic algorithm outperforms path scanning on all CARP instances, though by smaller margins than the CCPP instances, SPLICE also outperforms path scanning by almost 5% more than the memetic algorithm.

4.4.3. Results of the generalisation experiment

To investigate the generalisation capability of SPLICE, we first trained the model for 10,000 episodes using the smaller CARP instances, i.e., with 15 edges and with a vehicle capacity of $C = 8$; see §4.3. Then the resulting model is tested on ten instances selected uniformly at random from the CARP data set, i.e., with 29 edges but with the same vehicle capacity of $C = 8$.

For each of these instances, Table 4 gives the total cost of the solutions achieved by SPLICE (S) and path scanning (PS), the percentage decrease in total cost of the SPLICE solution compared to the path scanning solution $(1 - \frac{S}{PS}) \times 100\%$, the total cost of the solution achieved by the memetic algorithm (MA), and the percentage decrease in total cost of the SPLICE solution compared to the memetic algorithm solution $(1 - \frac{S}{MA}) \times 100\%$. Also given is the average decrease in total cost over the ten instances for the heuristic and the memetic algorithm. The total cost for these ten instances is also plotted in Fig. 7.

As Table 4 and Fig. 7 show, SPLICE appears to generalize well. SPLICE produces solutions with 46.60% and 39.62% lower total cost on average compared to path scanning and the memetic algorithm, respectively. As in the CCPP and CARP experiments, the memetic algorithm outperforms path scanning on all instances.

4.4.4. Experiment summary

A box and whisker plot is a graphical representation of the minimum, first quartile, median, third quartile, and the maximum, of a set of data. It graphically demonstrates the locality, spread and skewness of a set of data through their quartiles.

Table 5 and Fig. 8 give the values and the corresponding box and whisker plot, respectively, of the percentage of reduction in the total

Table 4
Generalisation results. The total cost for ten instances of CARP selected at random from 2,000 test instances from the CARP data set for SPLICE (S), path scanning (PS), and the memetic algorithm (MA) using a model trained on smaller instances. In addition, the percentage decrease in total cost of the SPLICE solution compared to that of path scanning (%↓ $\frac{S}{PS}$), and compared to that of the memetic algorithm (%↓ $\frac{S}{MA}$).

	Total Cost				
Instance	S	PS	%↓ $\frac{S}{PS}$	MA	%↓ $\frac{S}{MA}$
29E_2185	241	440	45.23	374	35.56
29E_5093	253	490	48.37	429	41.03
29E_155	266	524	49.24	463	42.55
29E_9689	268	525	48.95	490	45.31
29E_7372	250	445	43.82	425	41.18
29E_15979	259	460	43.70	412	37.14
29E_14782	264	545	51.56	460	42.61
29E_11832	260	536	51.49	461	43.60
29E_776	245	379	35.36	325	24.62
29E_4620	233	451	48.34	406	42.61
		Avg:	46.60	*Avg:*	39.62

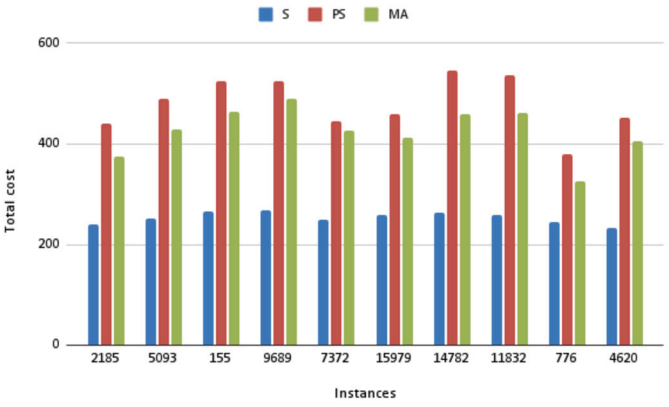


Fig. 7. Generalisation results. The total cost of the solutions for the ten instances of CARP from Table 4 with 29 edges and $C = 8$ for SPLICE (S), the path scanning heuristic (PS), and the memetic algorithm meta-heuristic (MA).

cost of solutions found by SPLICE with that of the path scanning heuristic algorithm for the CCPP, CARP, and generalisation experiments. Table 6 and Fig. 9 provide the same results but for the percentage of reduction in the total cost of solutions found by SPLICE with that of the memetic algorithm meta-heuristic.

These results follow from the expectation that meta-heuristics generally outperform heuristics (Prins, 2013). All values are lower for the meta-heuristic algorithm than for the heuristic algorithm.

4.5. *The computational and training costs of* SPLICE

Training SPLICE on 10,000 episodes where each batch of 16 instances has 29 edges takes less than 20 minutes of CPU time using the cloud resources from Google. At that time, the resources offered were Intel Xeon CPUs with and 4 GB of RAM. Once trained, an inference on an instance of the same size takes less than a second. Table 7 shows the CPU time used for inference on a CCPP instance, a CARP instance, and a smaller CARP instance used for training in the generalisation experiment. The inference time for SPLICE for each of the three experiments, each with 10 instances, is around 0.8 seconds.

As Fig. 5 shows, we obtained 23.43% reduction in total cost for CCPP instances using SPLICE but the running time was a fraction of a second (0.86 seconds versus over 53 seconds for the memetic algorithm). Similarly we obtained an average of 10.75% reduction in total

Table 5
For each of the CCPP, CARP, and generalisation experiment, the minimum, first quartile, median, third quartile, and the maximum, percentage reduction in total cost of SPLICE compared to path scanning.

Exp.	Min	Q1	Median	Q3	Max
CCPP	17.04	25.50	29.76	33.00	37.70
CARP	6.01	10.86	17.68	18.71	27.16
Gen.	35.36	44.17	48.36	49.17	51.56

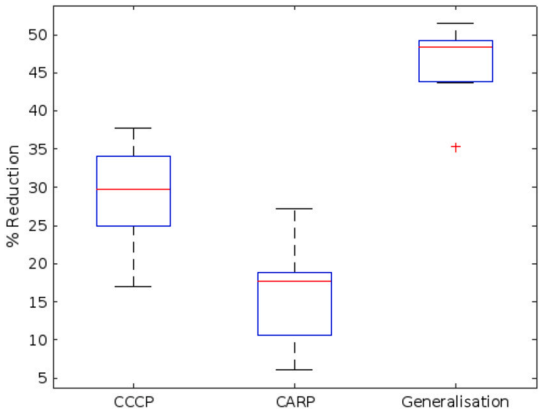


Fig. 8. A box and whiskers plot of the results in Table 5 indicating the percentage of reduction in total cost for the CCPP, CARP, and generalisation experiments compared to path scanning.

Table 6
For each of the CCPP, CARP, and generalisation experiment, the minimum, first quartile, median, third quartile, and the maximum, percentage reduction in total cost of SPLICE compared to the memetic algorithm.

Exp.	Min	Q1	Median	Q3	Max
CCPP	11.81	22.79	24.00	26.77	30.91
CARP	-4.33	7.98	11.90	17.32	20.00
Gen.	24.62	38.11	41.87	42.61	45.31

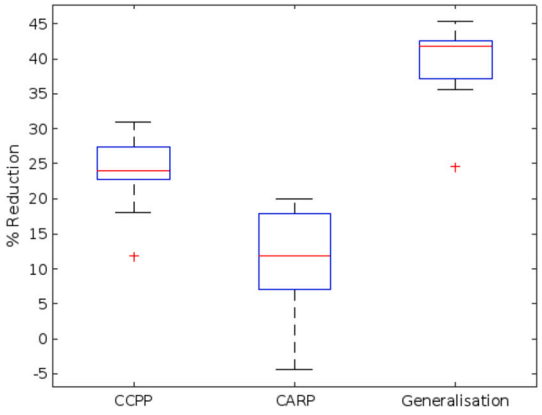


Fig. 9. A box and whiskers plot of the results in Table 6 indicating the percentage of reduction in total cost for the CCPP, CARP, and generalisation experiments compared to the memetic algorithm.

cost for CARP instances using 0.82 seconds versus 41.70 seconds for MA. For the generalisation experiment, we obtained around a 39.62% reduction in total cost with around the same inference time (0.8 seconds versus 57.3 seconds for MA).

The results show that the run time of the path scanning heuristic is faster than SPLICE. This is not surprising because path scanning con-

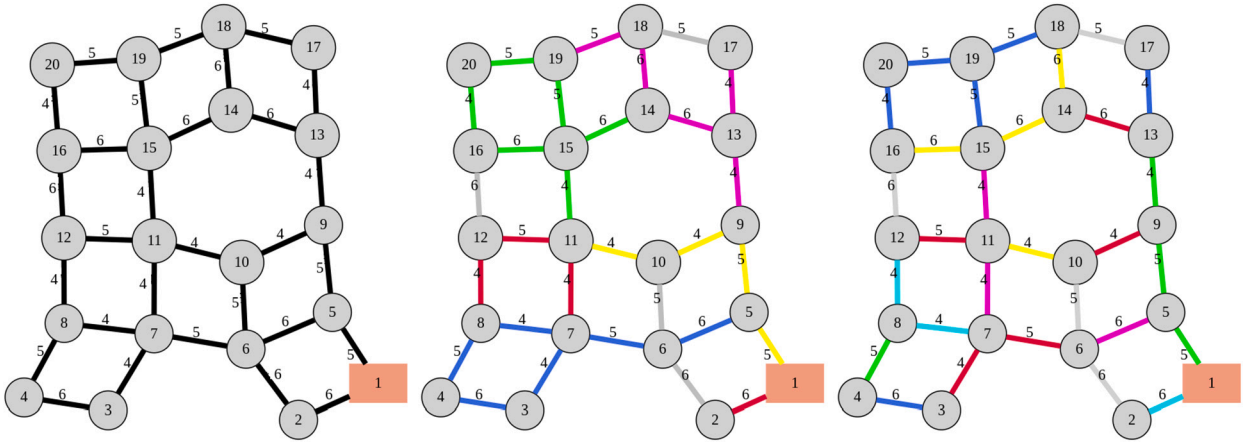


Fig. 10. A CARP problem instance (left), routes found by MA (centre) having a total cost of 429, and routes found by SPLICE with total cost of 253 (right). The grey edges are not required; i.e., they are only used for deadheading. While the routes found by SPLICE have lower total cost than those found by MA, they are not considered attractive because they are not contiguous.

Table 7
The average CPU time in seconds taken for running 10 instances at inference by SPLICE, path scanning (PS), and the memetic algorithm (MA).

	CPU Time		
Instance	SPLICE	PS	MA
CCPP, 29 edges, $C = 16$	0.86 s	0.37 s	53.16 s
CARP, 29 edges, $C = 16$	0.82 s	0.26 s	41.70 s
CARP, 29 edges, $C = 8$	0.80 s	0.28 s	57.33 s

structs the solution by selecting edges in a greedy fashion and hence the run time is expected to be faster. However, the solution quality is lower, as is evident from Figs. 5, 6, and 7.

SPLICE enables the same architecture to be used for different variants and in some cases with the same hyper-parameters or without the necessity of re-tuning the hyper parameters. This allows for substantial savings on training costs. For example, we used the same hyper-parameters without re-tuning for the three experiments, CCPP, CARP and for generalisation.

SPLICE uses a simple message passing GNN and feed forward layers. We intentionally avoided using sequential layers like RNN, LSTM which were known to be slower in processing. We also avoided using attention mechanisms and transformers, as they are complex, and take more time when not using the sophisticated parallel processing infrastructure and GPUs.

These computational and training costs demonstrate the utility of SPLICE for inference, and also its potential usefulness for changes in operational instances.

5. Discussion

From the results presented in §4.4, the SPLICE framework is effective in learning the distribution $\mathcal{D}$ from which instances are drawn, and yields improved total cost, performing better than the memetic algorithm (MA) and the path scanning (PS) heuristic, in a reasonable running time. The memetic algorithm is a hybrid evolutionary algorithm, combining evolutionary computing with local search. It can be thought of an algorithm that is learning to route using evolutionary algorithms but is unable to leverage its past learning in a data-driven manner. The memetic algorithm (Lacomme et al., 2004) also uses the route-first split-second splitting procedure. However, because SPLICE is able to learn the parameters of the distribution $\mathcal{D}$ it is able to give better results. The generalisation results in Fig. 7 shows that SPLICE generalizes well, able to scale for a change in the problem size.

An advantage of machine learning over meta-heuristic methods is fast inference. As the results on total cost show, SPLICE performs better than both MA and PS because of its capability to learn the distribution. At the same time the running time for SPLICE is from 50 to 72 times faster than MA. The model may be useful for and of potential interest for operators that experience changes in problem specifications.

In their daily operations, operators of services frequently encounter changes in the problem instance. For example in solid waste collection, operators encounter vehicle breakdowns resulting in a change in the availability of vehicles (Ramamoorthy & Syrotiuk, 2020), a change in the streets to service due to events such as local festivals, and a change in demand and cost of servicing the streets due to holidays and seasonal load. A machine learning based approach would be useful to find a cost effective solution in such scenarios because the changed problem instance is consistent with the distribution already learned. Thus a solver can generate a high quality solution more efficiently than a heuristic or meta-heuristic approach.

The SPLICE framework is designed for optimizing one objective, the total cost. Since SPLICE is effective in learning the distribution $\mathcal{D}$ in a data-driven manner, we obtain from 10% to 40% improvement in total cost over the memetic algorithm. However since it is trained only for the objective of minimum total cost, the generated routes are not considered attractive (i.e., the routes are not compact nor contiguous) (Ramamoorthy et al., 2021); see Fig. 10 for an example of the solutions found by each of these methods.

The SPLICE framework may be applied to variants of both node and arc routing problems. Variants such as the multi-depot arc routing problem, the split-delivery capacitated arc routing problem, and others, can be accommodated using a splitting procedure that corresponds to the variant (Corberán & Laporte, 2015, Mourão & Pinto, 2017). The SPLICE network architecture requires no other changes. Its simple design, which produces a single tour, simplifies the incorporation of new innovations in machine learning methods into the architecture.

5.1. *Limitations of* SPLICE

There are two types of heuristics: Constructive heuristics and improvement heuristics. Constructive heuristics build a single solution starting from an empty solution. Improvement heuristics start from a given initial solution and improve it by iteratively looking for a better solution using local search operations. Such heuristics typically take longer to execute than constructive heuristics, and often result in better solutions. However such methods cannot be guaranteed to find the global optimum and can get stuck in a local optimum.

SPLICE learns constructive heuristics. We choose this option in order to have fast inference. However, since SPLICE learns the distribution of the generating function during the training phase it can generate a closer to optimum solution at the time of inference.

Specifically, SPLICE learns heuristics of the form route-first split-second. As a result, the SPLICE framework cannot be applied to other kinds of combinatorial optimisation problems such as graph colouring, max cut, independent set maximisation, and others.

It is also the case that a giant tour of minimum cost that is split into a routing solution does not necessarily result in a solution of minimum total cost. SPLICE does not learn to construct the tour of minimum cost. Instead, it learns a tour that yields minimum total cost upon optimal splitting.

Machine learning methods cannot be expected to produce the same quality of results for problems that are from a different generating function or apply to problems that lie outside of the distribution of the training data set. Hence the availability of the real-world data sets, and associated training costs, may be a limitation of a framework such as SPLICE.

6. Conclusions and future work

We developed a deep Q-learning based framework SPLICE that learns heuristics for the capacitated arc routing problem (CARP). It uses a message passing based graph neural network to learn the graph structure representation of the problem and, from the graph embedding, learns heuristics using Q-learning.

SPLICE yields improved results over a memetic algorithm metaheuristic with an average 10% to 24% lower total cost with the inference running in around 0.8 seconds. SPLICE also generalizes well. These results demonstrate that SPLICE has the potential to be useful for real operational requirements.

SPLICE can be applied to different variants of arc routing because it learns route-first split-second heuristics. It can also be applied to node routing problems and its variants by tailoring the splitting procedure.

SPLICE learns constructive heuristics. Leveraging it to learn improvement heuristics is an interesting research area that we plan to explore. In addition, we plan to improve SPLICE to be able to solve for routing under breakdown scenarios (Ramamoorthy & Syrotiuk, 2020), where the problem involves routing with fewer vehicles, with each route starting from different location. We also plan to improve SPLICE to be able to solve routing problems with multiple objectives such as route balance and attractiveness (Ramamoorthy et al., 2021). One promising area to explore is *transfer learning* in which a model trained for one variant can be fine-tuned to solve a different variant with limited samples or training ("few shot learning"). Other interesting directions to explore include stochastic variants of CARP (Corberán & Laporte, 2015, Mourão & Pinto, 2017) and extending SPLICE to solve *node, edge, arc routing problems* (NEARP) (Prins & Bouchenoua, 2005, Vidal, 2017), which is a routing problem defined on mixed graphs with demands requiring services located on nodes, edges, and arcs.

CRediT authorship contribution statement

Muhilan Ramamoorthy: Conceptualization, Methodology, Software, Validation, Writing – original draft, Writing – review & editing. **Violet R. Syrotiuk:** Conceptualization, Supervision, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

Acknowledgements

We thank the anonymous referees for their useful comments and questions which have improved the quality of our paper.

Appendix A. Supplementary material

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.iswa.2023.200300.

References

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *3rd international conference on learning representations, ICLR 2015*.

Balcan, M.-F., Dick, T., Sandholm, T., & Vitercik, E. (2018). Learning to branch. In J. Dy, & A. Krause (Eds.), *Proceedings of machine learning research, PMLR: Vol. 80. Proceedings of the 35th international conference on machine learning* (pp. 344–353).

Barrett, T., Clements, W., Foerster, J., & Lvovsky, A. (2020). Exploratory combinatorial optimization with reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence: Vol. 34* (pp. 3243–3250).

Belenguer, J. M., Benavent, E., & Irnich, S. (2013). The capacitated arc routing problem: Exact algorithms. In *Arc routing: Problems, methods, and applications* (pp. 183–221). SIAM.

Bello, I., Pham, H., Le, Q. V., Norouzi, M., & Bengio, S. (2016). Neural combinatorial optimization with reinforcement learning. arXiv preprint. Retrieved from arXiv:1611.09940.

Bengio, Y., Lodi, A., & Prouvost, A. (2020). Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*.

Beullens, P., Muyldermans, L., Cattrysse, D., & Oudheusden, D. V. (2003). A guided local search heuristic for the capacitated arc routing problem. *European Journal of Operational Research*, *147*, 629–643.

Brandão, J., & Eglese, R. (2008). A deterministic tabu search algorithm for the capacitated arc routing problem. *Computers & Operations Research*, *35*, 1112–1126.

Bresson, X., & Laurent, T. (2017). Residual gated graph convnets. arXiv preprint. Retrieved from arXiv:1711.07553.

Chen, X., & Tian, Y. (2019). Learning to perform local rewriting for combinatorial optimization. In *Advances in neural information processing systems* (pp. 6281–6292).

Corberán, Á., & Laporte, G. (2015). *Arc routing: Problems, methods, and applications*. SIAM.

Corberán, A., & Prins, C. (2010). Recent results on arc routing problems: An annotated bibliography. *Networks*, *56*, 50–69.

Corberán, Á., Eglese, R., Hasle, G., Plana, I., & Sanchis, J. M. (2021). Arc routing problems: A review of the past, present, and future. *Networks*, *77*, 88–115.

Dai, H., Dai, B., & Song, L. (2016). Discriminative embeddings of latent variable models for structured data. In *International conference on machine learning* (pp. 2702–2711).

de O. Costa, P. R., Rhuggenaath, J., Zhang, Y., & Akcay, A. (2020). Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning. In *Asian conference on machine learning, PMLR* (pp. 465–480).

Defferrard, M., Bresson, X., & Vandergheynst, P. (2016). Convolutional neural networks on graphs with fast localized spectral filtering. In *Proceedings of the 30th international conference on neural information processing systems* (pp. 3844–3852).

Delarue, A., Anderson, R., & Tjandraatmadja, C. (2020). Reinforcement learning with combinatorial actions: An application to vehicle routing. *Advances in Neural Information Processing Systems*, *33*, 609–620.

Deudon, M., Cournut, P., Lacoste, A., Adulyasak, Y., & Rousseau, L.-M. (2018). Learning heuristics for the tsp by policy gradient. In *International conference on the integration of constraint programming, artificial intelligence, and operations research* (pp. 170–181). Springer.

Dror, M. (2012). *Arc routing: Theory, solutions and applications*. Springer Science & Business Media.

Durbin, R., & Willshaw, D. (1987). An analogue approach to the travelling salesman problem using an elastic net method. *Nature*, *326*, 689–691.

Eiselt, H. A., Gendreau, M., & Laporte, G. (1995). Arc routing problems, part I: The Chinese postman problem. *Operations Research*, *43*, 231–242.

Fedus, W., Ramachandran, P., Agarwal, R., Bengio, Y., Larochelle, H., Rowland, M., & Dabney, W. (2020). Revisiting fundamentals of experience replay. In *International conference on machine learning, PMLR* (pp. 3061–3071).

Fort, J. (1988). Solving a combinatorial problem via self-organizing process: An application of the Kohonen algorithm to the traveling salesman problem. *Biological cybernetics*, *59*, 33–40.

Gao, L., Chen, M., Chen, Q., Luo, G., Zhu, N., & Liu, Z. (2020). Learn to design the heuristics for vehicle routing problem. arXiv preprint. Retrieved from arXiv:2002.08539.

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In *International conference on machine learning, PMLR* (pp. 1263–1272).

Golden, B. L., & Wong, R. T. (1981). Capacitated arc routing problems. *Networks*, *11*, 305–315.

Golden, B. L., DeArmon, J. S., & Baker, E. K. (1983). Computational experiments with algorithms for a class of routing problems. *Computers & Operations Research*, *10*, 47–59.

Gross, J. L., Yellen, J., & Anderson, M. (2018). *Graph theory and its applications*. Chapman and Hall/CRC.

Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 855–864).

Hagberg, A., Swart, P., & Chult, D. S. (2008). Exploring network structure, dynamics, and function using NetworkX. Technical Report, Los Alamos, NM (United States): Los Alamos National Lab. (LANL).

Hamilton, W. L. (2020). Graph representation learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, *14*, 1–159.

Hamza, A., Darwish, A. H., & Rihawi, O. (2023). A new local search for the bees algorithm to optimize multiple traveling salesman problem. *Intelligent Systems with Applications*, Article 200242.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).

He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision* (pp. 2961–2969).

Hertz, A., & Mittaz, M. (2001). A variable neighborhood descent algorithm for the undirected capacitated arc routing problem. *Transportation science*, *35*, 425–434.

Hong, W., & Liu, T. (2022). Faster capacitated arc routing: A sequence-to-sequence approach. *IEEE Access*, *10*, 4777–4785.

Hopfield, J. J., & Tank, D. W. (1985). "Neural" computation of decisions in optimization problems. *Biological cybernetics*, *52*, 141–152.

Joshi, C. K., Laurent, T., & Bresson, X. (2019). An efficient graph convolutional network technique for the travelling salesman problem. arXiv preprint. Retrieved from arXiv: 1906.01227.

Khalil, E., Dai, H., Zhang, Y., Dilkina, B., & Song, L. (2017). Learning combinatorial optimization algorithms over graphs. *Advances in neural information processing systems*, *30*, 6348–6358.

Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., & Dilkina, B. (2016). Learning to branch in mixed integer programming. In *Thirtieth AAAI conference on artificial intelligence*.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In Y. Bengio, & Y. LeCun (Eds.), *3rd international conference on learning representations*. Retrieved from http://arxiv.org/abs/1412.6980.

Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! In *International conference on learning representations*.

Kool, W., van Hoof, H., Gromicho, J., & Welling, M. (2022). Deep policy dynamic programming for vehicle routing problems. In *Integration of constraint programming, artificial intelligence, and operations research* (pp. 190–213). Springer International Publishing.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, *25*.

Kwon, Y.-D., Choo, J., Kim, B., Yoon, I., Gwon, Y., & Min Pomo, S. (2020). Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems*, *33*, 21188–21198.

Lacomme, P., Prins, C., & Ramdane-Cherif, W. (2004). Competitive memetic algorithms for arc routing problems. *Annals of Operations Research*, *131*, 159–185.

Li, H., & Li, G. (2019). Learning to solve capacitated arc routing problems by policy gradient. In *2019 IEEE congress on evolutionary computation (CEC)* (pp. 1291–1298). IEEE.

Lovász, L. (1993). Random walks on graphs. *Combinatorics, Paul erdos is eighty*, *2*, 4.

Ma, N., Yin, H., & Wang, K. (2023). Prediction of the remaining useful life of supercapacitors at different temperatures based on improved long short-term memory. *Energies*, *16*, 5240.

Ma, P., Cui, S., Chen, M., Zhou, S., & Wang, K. (2023). Review of family-level short-term load forecasting and its application in household energy management system. *Energies*, *16*, 5809.

Marinaki, M., Taxidou, A., & Marinakis, Y. (2023). A hybrid dragonfly algorithm for the vehicle routing problem with stochastic demands. *Intelligent Systems with Applications*, Article 200225.

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing atari with deep reinforcement learning. arXiv preprint. Retrieved from arXiv:1312.5602.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*, 529–533.

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In *International conference on machine learning* (pp. 1928–1937).

Mourão, M. C., & Pinto, L. S. (2017). An updated annotated bibliography on arc routing problems. *Networks*, *70*, 144–194.

Muyldermans, L., & Pang, G. (2015). Variants of the capacitated arc routing problem. In *Arc routing: Problems, methods, and applications* (pp. 223–253). SIAM.

Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th international conference on international conference on machine learning* (pp. 807–814).

Nazari, M., Oroojlooy, A., Takáč, M., & Snyder, L. V. (2018). Reinforcement learning for solving the vehicle routing problem. In *Proceedings of the 32nd international conference on neural information processing systems* (pp. 9861–9871).

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, & R. Garnett (Eds.), *Advances in neural information processing systems: Vol. 32* (pp. 8024–8035). Curran Associates, Inc. Retrieved from http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf.

Pearn, W. L. (1989). Approximate solutions for the capacitated arc routing problem. *Computers & Operations Research*, *16*, 589–600.

Prins, C. (2013). The capacitated arc routing problem: Heuristics. In *Arc routing: Problems, methods, and applications* (pp. 131–157). SIAM.

Prins, C., & Bouchenoua, S. (2005). A memetic algorithm solving the VRP, the CARP and general routing problems with nodes, edges and arcs. In *Recent advances in memetic algorithms* (pp. 65–85). Springer.

Prins, C., Labadi, N., & Reghioui, M. (2009). Tour splitting algorithms for vehicle routing problems. *International Journal of Production Research*, *47*, 507–535.

Prins, C., Lacomme, P., & Prodhon, C. (2014). Order-first split-second methods for vehicle routing problems: A review. *Transportation Research Part C: Emerging Technologies*, *40*, 179–200.

Ramamoorthy, M. (2022). Heuristics for arc routing problems and their applications. Ph.D. thesis, Arizona State University.

Ramamoorthy, M., & Syrotiuk, V. R. (2020). Online re-routing for vehicle breakdown in residential waste collection. In *2020 IEEE 92nd vehicular technology conference (VTC2020-fall)* (pp. 1–5). IEEE.

Ramamoorthy, M., Forrest, S., & Syrotiuk, V. R. (2021). Ma-abc: A memetic algorithm optimizing attractiveness, balance, and cost for capacitated arc routing problems. In *Proceedings of the genetic and evolutionary computation conference* (pp. 1043–1051).

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 779–788).

Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, *28*.

Riedmiller, M. (2005). Neural fitted q iteration–first experiences with a data efficient neural reinforcement learning method. In *European conference on machine learning* (pp. 317–328). Springer.

Smith, K. A. (1999). Neural networks for combinatorial optimization: A review of more than a decade of research. *INFORMS Journal on Computing*, *11*, 15–34.

Sun, X., Zhang, Y., Zhang, Y., Wang, L., & Wang, K. (2023). Summary of health-state estimation of lithium-ion batteries based on electrochemical impedance spectroscopy. *Energies*, *16*, 5682.

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, *27*, 3104–3112.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT Press.

Ulusoy, G. (1985). The fleet size and mix problem for capacitated arc routing. *European Journal of Operational Research*, *22*, 329–337.

Usberti, F. L., França, P. M., & França, A. L. M. (2013). Grasp with evolutionary path-relinking for the capacitated arc routing problem. *Computers & Operations Research*, *40*, 3206–3217.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (pp. 5998–6008).

Vidal, T. (2016). Split algorithm in o (n) for the capacitated vehicle routing problem. *Computers & Operations Research*, *69*, 40–47.

Vidal, T. (2017). Node, edge, arc routing and turn penalties: Multiple problems—one neighborhood extension. *Operations Research*, *65*, 992–1010.

Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. *Advances in neural information processing systems*, *28*, 2692–2700.

Watkins, C. (1989). Learning form delayed rewards. Ph.D. thesis, King's College, University of Cambridge.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, *8*, 229–256.

Wøhlk, S. (2005). Contributions to arc routing. Ph.D. thesis, Aarhus, Denmark: University of Southern Denmark.

Wu, Y., Song, W., Cao, Z., Zhang, J., & Lim, A. (2019). Learning improvement heuristics for solving the travelling salesman problem. arXiv preprint. Retrieved from arXiv: 1912.05784.

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? In *International conference on learning representations*.

Yaddaden, A., Harispe, S., & Vasquez, M. (2022). Neural order-first split-second algorithm for the capacitated vehicle routing problem. In *Optimization and learning: 5th international conference* (pp. 168–185). Springer.

Yi, Z., Chen, Z., Yin, K., Wang, L., & Wang, K. (2023). Sensing as the key to the safety and sustainability of new energy storage devices. *Protection and Control of Modern Power Systems*, *8*, 1–22.

Zhang, S., & Sutton, R. S. (2017). A deeper look at experience replay. arXiv preprint. Retrieved from, arXiv:1712.01275.